

RDM — Design History File

Document ID	README
Revision	395af08
Date	2026-10-03
Status	Controlled in git

Contents

1 Layout (the record)	3
2 docs/ and dhf/	4
3 Agents in this repository	4

This is RDM's own DHF: RDM is developed under RDM's own design controls, and this directory is the record. Every change to RDM goes through the loop RDM asks of a device team: record first, then a tagged test, then the gates, then a reviewed pull request. The procedure is `AGENT_WORKFLOW.md` (this directory); the method is the docs' *How RDM works* (`docs/about/how-rdm-works.md`), and the decision behind user needs and bounded contexts is ADR 0001 (`decisions/`).

Scope note: RDM is not a medical device, so this DHF deliberately implements the **design-controls slice** (design inputs, review, verification, validation, traceability — the §820.30-shaped record) plus a register of RDM's own tool risks, and not the full IEC 62304 lifecycle document set (a complete risk management file, SOUP register, maintenance and problem-resolution plans, ...). Running `rdm gap 62304_2015_class_b` over these documents is expected to report those items as missing.

1 Layout (the record)

<code>AGENT_WORKFLOW.md</code>	the change procedure (start here)
<code>documents/</code>	
<code>design_review.md</code>	design review record (gated)
<code>verification_and_validation_plan.md</code>	user-need registry (UN-...) + V&V approach
<code>document_control.md</code>	git/GitHub as this record's document control
(Part 11-mapped)	
<code>architecture.md</code>	system design / bounded contexts (design only, no needs)
<code>traceability_matrix.md</code>	matrix TEMPLATE (rendered from generated data; never hand-edited)
<code>design/</code>	one `kind: design` doc per bounded context:
<code><context>.md</code>	the design inputs it owns (what) + design output (how)
<code>risk/</code>	
<code>policy.md</code>	risk acceptability policy (RMP-001, proposed)
<code>tool_risks.md</code>	`kind: risk` register of RDM's tool risks
(RMF-001, proposed)	
<code>c4/</code>	the architecture as a Structurizr workspace, and its drawn views
<code>decisions/</code>	decision records (ADR 0001)

The live inventory — which contexts exist, which design inputs each owns, and which user needs they trace to — is generated, not maintained here:

```
rdm story new-input --dhf dhf --list    # contexts, taken DI ids, user needs
rdm story trace UN-... | DI-...        # one need's / input's slice
```

The gates, as CI runs them, are step 6 of `AGENT_WORKFLOW.md`. `dhf/allure-results/` and `dhf/data/verification.yml` are generated (gitignored) by running the acceptance suite, never committed.

2 docs/ and dhf/

docs/ explains RDM to its users; dhf/ is the controlled record of how RDM itself is designed and verified. One explains, the other decides.

	docs/	dhf/
What it is	RDM's user documentation, plus an API reference generated from the docstrings	RDM's own design history file: the regulated record RDM develops itself under
Who it's for	People and agents using RDM on their own product	Whoever governs RDM's development: authors, reviewers, an auditor, and RDM's gates
Controlled?	No: ordinary docs, changed by a normal reviewed pull request	Yes: the design gate requires it complete and committed before code; the release gate requires every design input in it verified
Checked by	mkdocs build --strict (links, navigation)	The design gate, the release gate, rdm graph validate, and the tests tagged with its ids
Read by RDM's tools?	No	Yes: the gates, trace, the graph, the verification report and the evidence bundle
Published	As the docs site	On the same site, each document as a page and a PDF, with the traceability matrix and map generated from a live test run at each build

Nothing flows from docs/ into the record. Where the docs would restate what the record decides, they link to it instead. A product that adopts RDM gets its own dhf/ (rdm adopt or rdm init), not RDM's docs/.

3 Agents in this repository

Agents working on RDM read the record through rdm graph mcp, registered in the repository's .mcp.json. A session's bootstrap script syncs dependencies and turns on the design-gate hook, so an agent works under the same gates as a person.